

# Distribuirani sistemi - Sinhronizacija

---

- Logički časovnici
- Uzajamno isključivanje
- Algoritmi izbora koordinatora

# Logički časovnici

- \* U mnogim aplikacijama nije važno fizičko vreme kada je neki događaj nastupio, već realtivni redosled događaja( npr. *make* progam)
  - dovoljno je da se mašine koje ostavruju međusobnu interakciju usaglasu oko realativnog vremena i to vreme ne mora da se slaže sa fizičkim vremenom.
    - Bitan je samo realtivni odnos događaja
  - Realtivno uređenje događaja bazira se na tzv. logičkim časovnicima.

# Lamportov algoritam sinhronizacije

\* Određuje redosled događaja, ali ne vrši sinhronizaciju časovnika.

- Koristi se za sinhronizaciju logičkih časovnika

\* Lamport je definisao relaciju “desilo se pre” (happened before)

- $A \longrightarrow B$  čita se “A se desilo pre B”.

- To znači da se svi procesi slažu da je prvo nastupio događaj A a zatim događaj B.

- Ova relacija može biti zadovoljena u dva slučaja

- Ako su  $a$  i  $b$  događaji u istom procesu, i ako  $a$  nastupa pre  $b$ , tada je relacija  $a \rightarrow b$  tačna.

- Ako je  $a$  događaj koji predstavlja slanje poruke iz jednog procesa, a  $b$  događaj prijema poruke u drugom procesu, tada je  $a \rightarrow b$  takođe tačna.

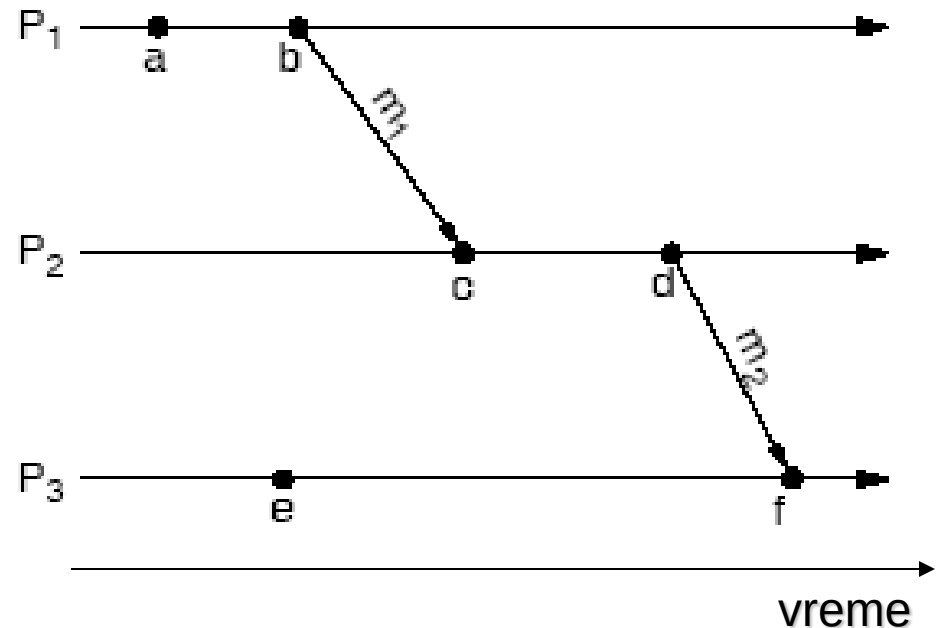
- Poruka ne može biti primljena pre nego što je poslata!

# Lamportov algoritam (nast.)

Ova operacija je tranzitivna:

$a \rightarrow b$ ,  $b \rightarrow c$ , tada vazi  $a \rightarrow c$

Ako su se događaji  $x$  i  $y$  desili u dva procesa koji ne razmenjuju poruke (čak ni posredno, preko treće strane) tada relacija  $x \rightarrow y$  ili  $y \rightarrow x$  ne važi, a za događaje  $x$  i  $y$  se kaže da su konkurentni



$P_1, P_2, P_3$ : processes;

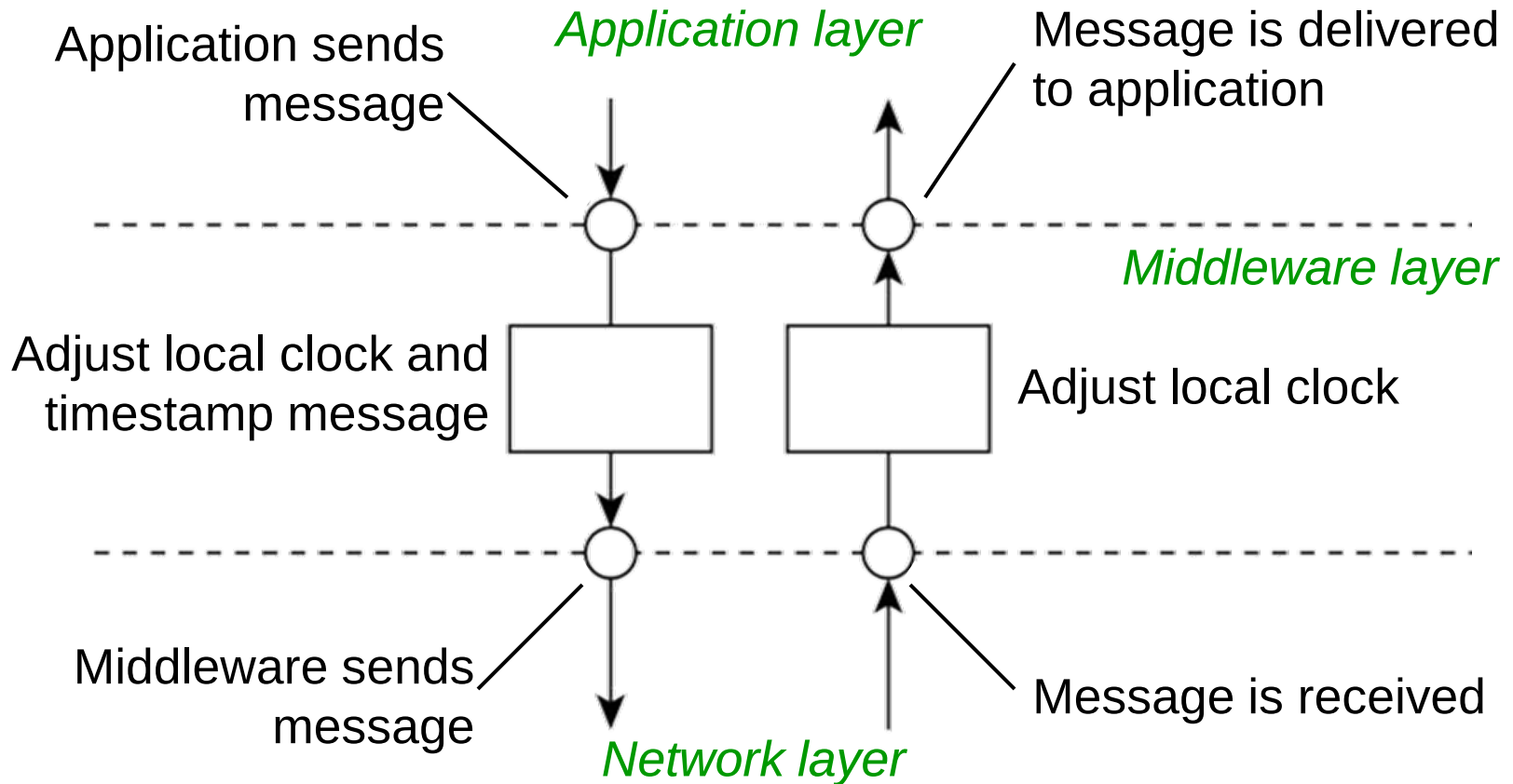
$a, b, c, d, e, f$ : events;

$a \rightarrow b, c \rightarrow d, e \rightarrow f, b \rightarrow c, d \rightarrow f$

$a \rightarrow c, a \rightarrow d, a \rightarrow f, b \rightarrow d, b \rightarrow f, \dots$

$a \parallel e, c \parallel e, \dots$

# Pozicioniranje Lamportovih logičkih časovnika u DS

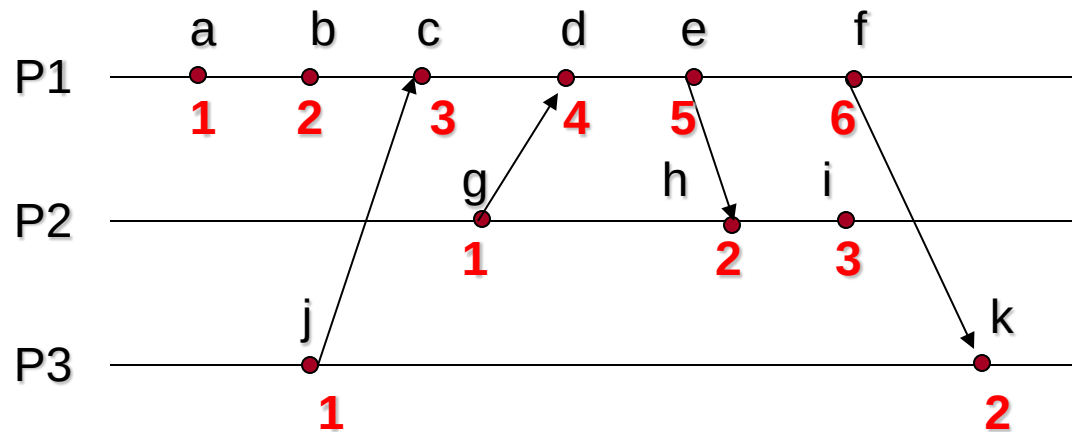


# Lamportov algoritam (nast.)

- \* Ako je realcija  $a \rightarrow b$  tačna, tada je potrebno ostvariti sinhronizaciju logičkih časovnika tako da važi da je  $T(a) < T(b)$ .
  - Svakom događaju se dodeljuje vremenska markica (timestamp)
    - Između svaka dva događaja sat mora da otkuca bar jednom
  - Sinhronizacija se ostvaruje korekcijom logičkih časovnika tako što se časovniku dodaje neka vrednost (vreme mora uvek da raste, ne može da se vraća u nazad)
- \* Pravila
  1. Logički časovnik  $L_i$  se inkrementira pre izvršenja bilo koje aktivnosti (slanje poruke kroz mrežu, isporuka poruke aplikaciji, ili neki drugi interni događaj) u procesu  $P_i$ ,  $L_i = L_i + 1$ ;
  2. Kada proces  $P_i$  šalje poruku  $m$ , on u nju ubacuje i vrednost  $t = L_i$ ;
  3. Kada proces  $P_j$  primi poruku  $(m, t)$ , on podešava svoj lokalni logički časovnik tako što određuje  $L_j = \max(L_j, t)$  i primenjuje pravilo 1, a zatim prosledjuje poruku aplikaciji

# Primer

- \* Tri procesa P1, P2 i P3 na tri mašine
- \* Događaji a, b, c, ...
- \* Lokalni brojač događaja na svakoj mašini
- \* Procesi povremeno razmenjuju poruke.



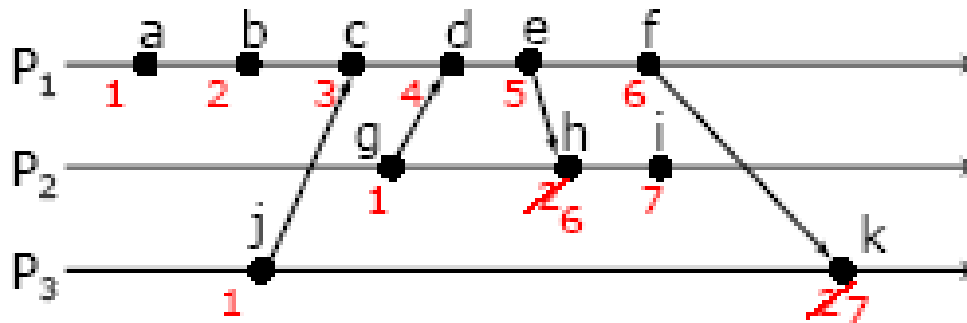
Loša uređenost:

$e \rightarrow h$ , ali je  $T(e) > T(h)$

$f \rightarrow k$ , ali je  $T(f) > T(k)$

# Lamportov algoritam

- \* Svaka poruka nosi vremensku markicu izvornog časovnika
- \* Kada poruka stigne proverava se
  - if lokalni\_sat < vremenska\_markica\_poruke then
  - postavi lokalni sat na (vremenska markica + 1)
  - else ništa nije potrebno korigovati
- Između bilo koja dva događaja sat mora otkucati bar jednom.

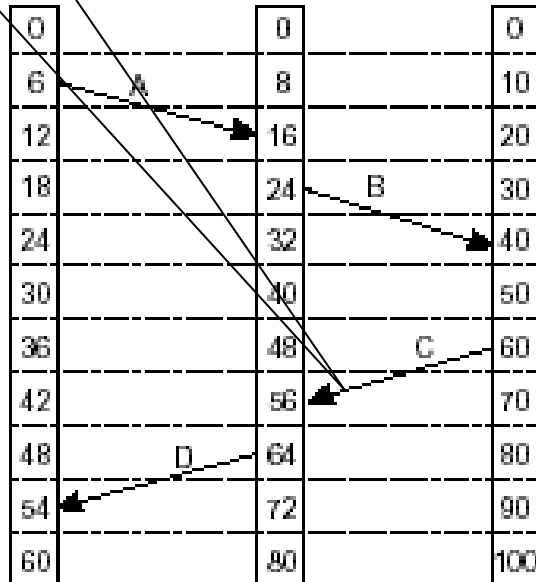


$$e \rightarrow h, \quad T(e) < T(h)$$
$$f \rightarrow k, \quad T(f) < T(k)$$

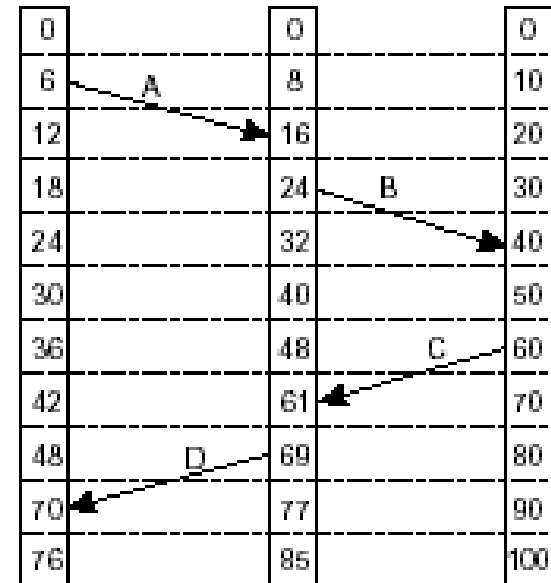
Algoritam omogućava da se održi vremensko uređenje međusobno uslovljenih događaja. (parcijalna uređenost unutar jednog procesa)

# Lamportove vremenske markice

poruka primljena pre  
nego što je poslata



(a)

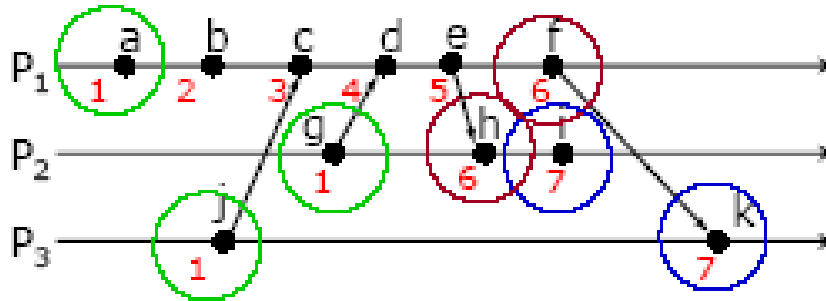


(b)

- a) Tri procesa sa svojim lokalnim satovima. Satovi rade različitim brzinama
- b) Lamportov algoritam vrši korekciju satova

# Problemi sa Lamportovim algoritmom\_1

- \* Moguće je da više događaja koji nisu međusobno uslovljeni (tj. konkurentni su) imaju identične vremenske markice



- Ovo može dovesti do konfuzije ako više procesa treba da donese odluku na osnovu vremenskih markica dva događaja.
  - Ako su događaji konkurentni, redosled nije bitan, ali potrebno je da svi procesi donesu istu odluku
    - To je teško postići ako su vremenske markice identične.

# Problemi sa Lamportovim algoritmom\_1 (nast.)

## \* Rešenje

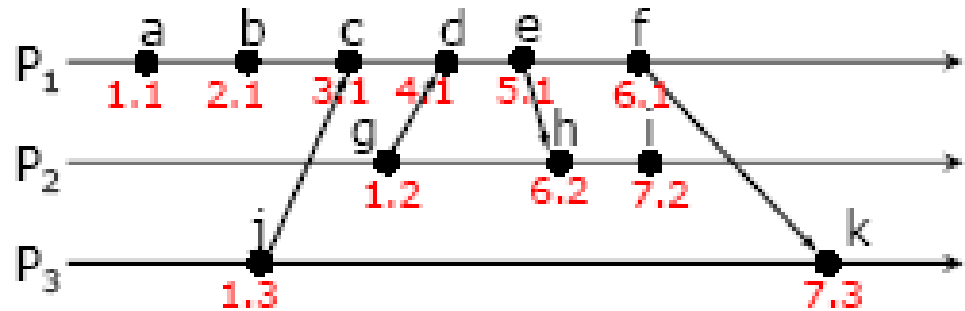
- Prisiliti da svaka markica bude jedinstvena.
  - Definisati globalnu vremensku markicu  $(T_i, i)$ 
    - $T_i$  predstavlja lokalnu Lamportovu markicu
    - $i$  predstavlja broj procesa (globalno jedinstven)
      - » Npr. adresa hosta, proces ID
- Poređenje markica

$$(T_i, i) < (T_j, j)$$

ako i samo ako

$$T_i < T_j, \text{ ili}$$

$$T_i = T_j, \text{ i } i < j$$



# Primer: potpuno uredjena grupna komunikacija

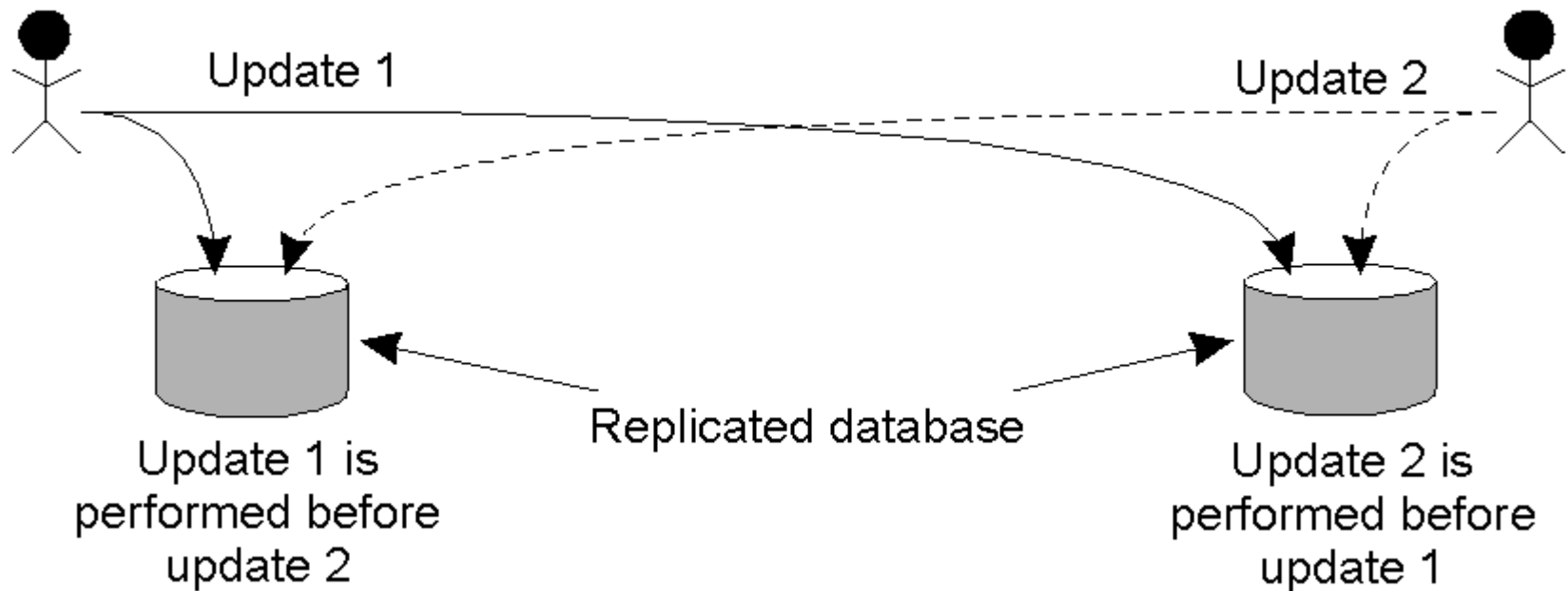
## \* Primena Lamportovih markica

## \* Scenario

- Replicirani bankovni računi u New Yorku (NY) i San Francisku (SF)
- Dve transakcije desavaju se jednovremeno i obavlja se multicast (zbog ažuriranja replika)
  - tekuće stanje: \$1,000
  - klijent dodaje \$100 na svoj račun u SF
  - službenik dodaje kamatu na račun od 1% u NY
  - ako ažuriranja računa nisu obavljena u istom redosledu u obe kopije (NY i SF) nakon ažuriranja jedna kopija će sadržati stanje računa \$1,111 a druga \$1,110.

# Primer: potpuno uredjena grupna komunikacija

- \* Ažuriranje replicirane baze podataka koje ostavlja kopije u nekonzistentnom stanju.



# Primer: potpuno uredjena grupna komunikacija

- \* Potrebno je obezbediti da se operacije ažuriranja obave na obe strane u istom redosledu.
  - Mada postoji razlika da li će se prvo obaviti zaračunavanje kamate pa dodavanje depozita ili obrnuto, sa stanovišta konzistentnosti kopija (replika) nije važno koji se redosled poštuje.
- \* Neophodno je obezbediti potpuno uredjenu grupnu komunikacija (multicast), tj. operaciju kojom se sve poruke isporučuju u istom redosledu svim prijemnicima.
  - NAPOMENA: Multicast se odnosi na slanje poruke iz jednog izvora u više odredišta (prijemnika).

# Primer: potpuno uredjena grupna komunikacija

## \* Algoritam

- Poruka ažuriranja se obeležava logičkim vremenom izvora
- Poruka ažuriranja se prosledjuje svima u grupi (multicast) uključujući i samog sebe
- Kada poruka stigne
  - smešta se u lokalni red čekanja (queue)
  - uredjuju se po markicama,
  - Poruka potvrde (ack) se prosledjuje svima (multicast) uključujući i samog sebe
- poruke iz istog izvora stižu u redosledu u kome su poslate (FIFO)

# Primer: potpuno uredjena grupna komunikacija

- \* Poruka se prosledjuje aplikaciji samo ako
  - se nalazi na vrhu reda
  - potvrđena je od strane svih procesa
- \*  $P_i$  šalje potvrdu (ack) za poruku koju je primio od  $P_j$  ako
  - $P_i$  nije poslao poruku ažuriranja
  - $P_i$  -ov identifikator je veći od identifikatora  $P_j$ -a
  - $P_i$ -ov zahtev za ažuriranjem je već obradjen;
- \* Lamportov algoritam garantuje totalno uredjenje dogadjaja

# Primer: potpuno uredjena grupna komunikacija

- \* Poruka m odgovara poruci “dodaj \$100” a n odgovara poruci “Dodaj kamatu od 1%”.
- \* Kada se šalju poruke ažuriranja (npr., m, n) poruka će sadržati markicu koja je generisana kada je ažuriranje izdato.
  - Slanje poruke m sastoji se od slanja poruke ažuriranja i vremena kada je zahtev za ažuriranjem izdat (u ovom slučaju to je trenutak 1.1)
  - Slanje poruke n sastoji se od slanja poruke ažuriranja i vremena kada je zahtev za ažuriranjem izdat (u ovom slučaju to je trenutak 1.2)

# Primer: potpuno uredjena grupna komunikacija

- \* Poruke se prosledjuju svim procesima u grupi uključujući i samog sebe.
  - usvajamo da poruka koju proces šalje samom sebi stiže trenutno (tj. nema kašnjenja)
- \* Nakon slanja poruka redovi čekanja u procesima sadrže sledeće poruke:
  - P1: (m,1.1), (n,1.2)
  - P2: (m,1.1), (n,1.2)
- \* P1 će proslediti svima u grupi (multicast) potvrdu prijema za poruku (m,1.1) ali ne i za (n,1.2).
  - Zašto? Identifikator procesa P1 je niži od identifikatora procesa P2, a P1 je izdao poruku ažuriranja
  - $1.1 < 1.2$
  - P1 neće poslati potvrdu za (n,1.2) dok se operacija *m* ne izvrši.
- \* P2 će proslediti svima u grupi potvrdu za (m,1.1) i (n,1.2)
  - Zašto? Identifikator procesa P2 je viši od identifikatora procesa P1
  - $1.1 < 1.2$

# Primer: potpuno uredjena grupna komunikacija

- \* Ako P2 primi  $(n,1.2)$  pre  $(m,1.1)$  da li on svima u grupi prosledjuje potvrdu za  $(n,1.2)$ ?
  - Da!
- \* Postavlja se pitanje kako P2 zna da postoje druga ažuriranja koja treba da budu obavljena pre njegovog?
  - Ne zna;
  - Proces P2 ne može da obavi ažuriranje definisano porukom  $(n,1.2)$  dok ne dobije potvrde od svih ostalih procesa, što u ovom primeru znači od procesa P1.
    - poruke iz jednog izvora uvek stižu po redosledu, što znači da ne može da se desi da ack iz P1 stigne pre njegove poruke  $(m, 1.1)$  koju je ranije poslao
- \* Da li proces P2 šalje svima u grupi potvrdu za  $(m,1.1)$  kada primi ovu?
  - Da , zato što je  $1.1 < 1.2$

# Primer: potpuno uredjena grupna komunikacija

- \* Do sada su poslate sledeće poruke :
  - P1 i P2 su izdali zahteve za ažuriranjem,  $(m,1.1)$  i  $(n,1.2)$ .
  - P1 je prosledio svima u grupi potvrdu za poruku  $(m,1.1)$ .
  - P2 je prosledio svima u grupi potvrdu za poruku  $(m,1.1)$ ,  $(n,1.2)$ .
- \* P1 i P2 su primili potvrde od svih procesa iz grupe za poruku  $(m,1.1)$ .
- \* Ažuriranje koje se zahteva porukom  $m$  može biti obavljeno i u P1 i u P2.

# Primer: potpuno uredjena grupna komunikacija

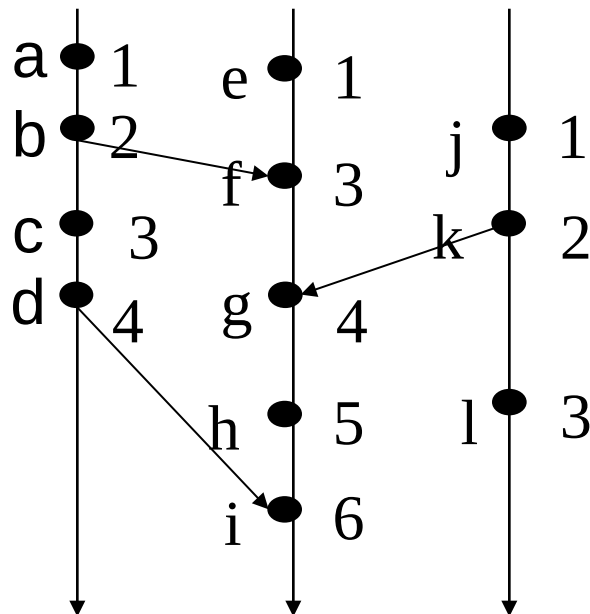
- \* Kada P1 završi sa ažuriranjem  $m$ , on šalje svim procesima u grupi potvrdu za  $(n, 1.2)$ .
- \* Kada potvrdu prime P1 i P2, znači da je potvrda za  $(n, 1.2)$  stigla od svih procesa iz grupe
- \* Sada ažuriranje koje se zahteva porukom  $n$  može da se obavi u oba procesa, P1 i P2.

# Primer: potpuno uredjena grupna komunikacija

- \* Šta da je postojao treći proces, npr. P3, koji je izdao zahtev za ažuriranjem ( $O, 1.3$ ) skoro u isto vreme kad i procesi P1 i P2?
- \* Algoritam funkcioniše na isti način:
  - P1 neće proslediti ostalim procesima u grupi potvrdu za  $O$ , dok se  $m$  ne obavi.
  - P2 neće poslati potvrdu ostalim procesima u grupi dok se  $n$  ne obavi.
- \* Pošto operacija ne može da se obavi dok iz svih procesa ne stignu potvrde, operacija ažuriranja  $O$  se neće obaviti dok se operacije  $m$  i  $n$  ne obave..

# Vektorski časovnici

- \* Lamportovim markicama se obezbedjuje da ako za dogadjaje  $a$  i  $b$  važi da je  $a \rightarrow b$ , tada važi  $T(a) < T(b)$
- \* Problem je što se na osnovu njih ne može utvrditi koji su dogadjaji medjusobno uslovljeni, a koji su konkurentni.
  - Ako važi da je  $T(a) < T(b)$  ne možemo zaključiti da se  $a$  desilo pre  $b$  (tj.  $a \rightarrow b$ ).



- Primer:  $T(e) = 1$  i  $T(b) = 2$ ; tj.  $T(e) < T(b)$  ali ne možemo reći da je  $e \rightarrow b$
- potreban nam je mehanizam za obeležavanje dogđaja takav da ako važi da je npr.  $T(a) < T(b)$ , tada možemo da zaključimo da  $a \rightarrow b$

# Vektorski časovnici

## \* Predloženo rešenje zasniva se na vektorskim časovnicima.

- Vektorski časovnik u sistemu sa  $N$  procesa je vektor od  $N$  celobrojnih elemenata.
- Svaki proces ima sopstveni vektorski časovnik kojim beleži lokalne događaje.
- Kao i Lamportove vremenske markice, i vektorski časovnik se šalje sa svakom porukom.

### ● Pravila:

1. Vektor se inicijalizuje na 0 u svim procesima:  $V_i[j]=0$ , za  $i,j=1,\dots,N$
2. Proces  $P_i$  inkrementira  $i$ -ti element vektora u lokalnom vektoru pre nego što obeleži događaj:  $V_i[i]=V_i[i]+1$ 
  - Poruka se šalje iz procesa  $P_i$  zajedno sa  $V_i$ .
3. Kada  $P_j$  primi poruku poredi lokalni vektor sa primljenim, element po element, i postavlja element u lokalnom vektoru na veću od vrednosti

$$V_j(k) = \max\{V_j(k), V_i(k)\}, \quad k = 1, 2, \dots, N$$

# Vektorski časovnici (nast.)

## \* Poređenje vektora

$$V = V' \text{ iff } V[i] = V'[i], \text{ za } i = 1, \dots, N$$

$$V \leq V' \text{ iff } V[i] \leq V'[i], \text{ za } i = 1, \dots, N$$

## \* Za bilo koja dva događaja $e$ i $e'$ važi sledeće

ako je  $e \rightarrow e'$  tada  $V(e) < V(e')$  (kao kod Lamportovog algoritma)

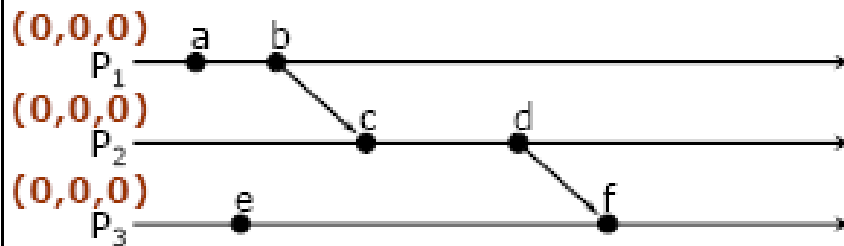
ako je  $V(e) < V(e')$  tada je  $e \rightarrow e'$

## \* Ako se ne može uspostaviti relacija $V(e) \leq V(e')$ ili $V(e) \geq V(e')$ , tada su događaji konkurentni (tj. nisu međusobno uslovljeni)

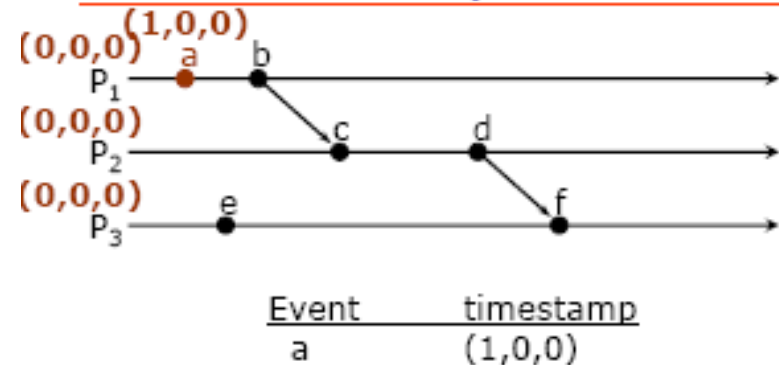
- Za proces  $P_i$ ,  $V_i[i]$  predstavlja broj događaja koji su se desili u procesu  $P_i$ ,
- Ako je  $V_i[j] = k$ , tada  $P_i$  zna da se u  $P_j$  desilo  $k$  događaja

# Primer

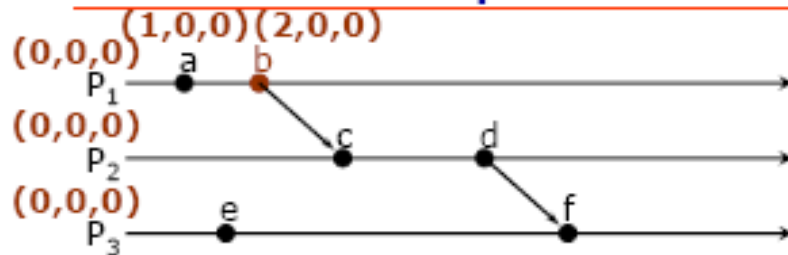
## Vector timestamps



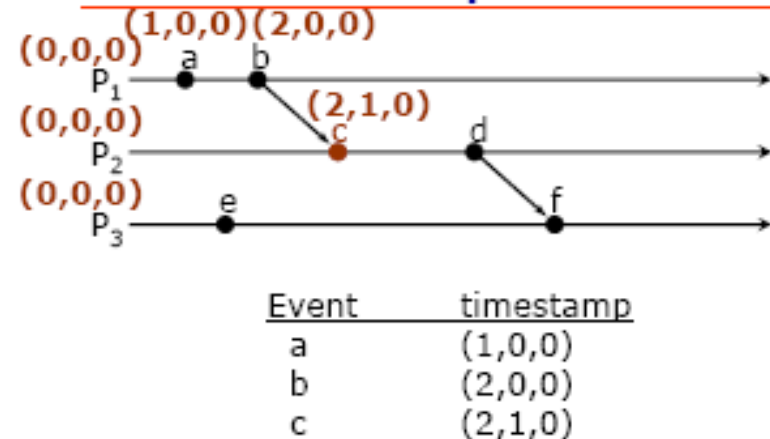
## Vector timestamps



## Vector timestamps

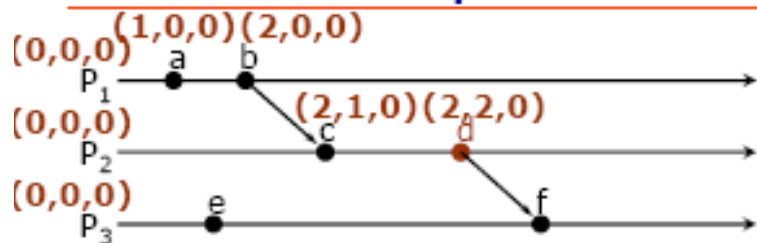


## Vector timestamps



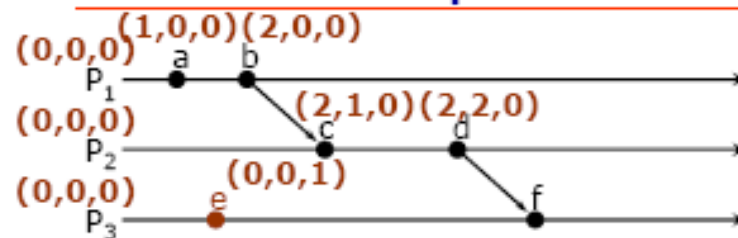
# Primer (nast.)

## Vector timestamps



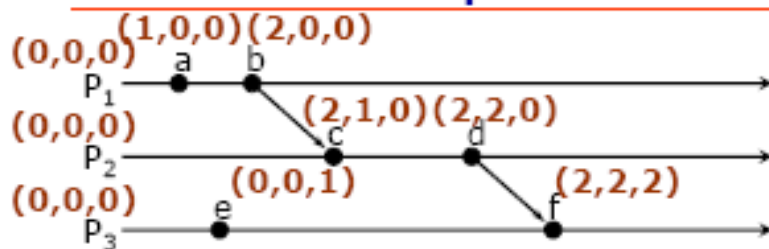
| Event | timestamp |
|-------|-----------|
| a     | (1,0,0)   |
| b     | (2,0,0)   |
| c     | (2,1,0)   |
| d     | (2,2,0)   |

## Vector timestamps



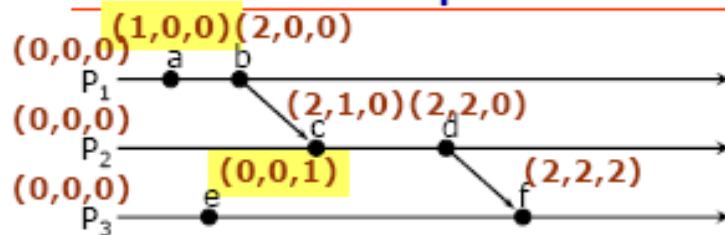
| Event | timestamp |
|-------|-----------|
| a     | (1,0,0)   |
| b     | (2,0,0)   |
| c     | (2,1,0)   |
| d     | (2,2,0)   |
| e     | (0,0,1)   |

## Vector timestamps



| Event | timestamp |
|-------|-----------|
| a     | (1,0,0)   |
| b     | (2,0,0)   |
| c     | (2,1,0)   |
| d     | (2,2,0)   |
| e     | (0,0,1)   |
| f     | (2,2,2)   |

## Vector timestamps

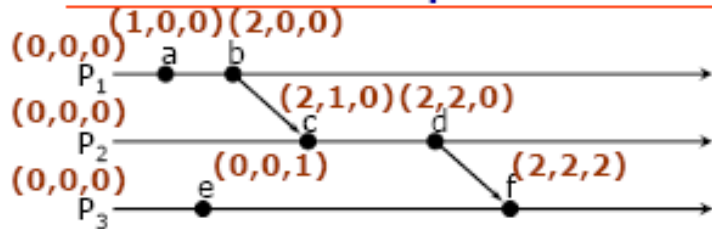


| Event | timestamp |
|-------|-----------|
| a     | (1,0,0)   |
| b     | (2,0,0)   |
| c     | (2,1,0)   |
| d     | (2,2,0)   |
| e     | (0,0,1)   |
| f     | (2,2,2)   |

concurrent events

# Primer (nast.)

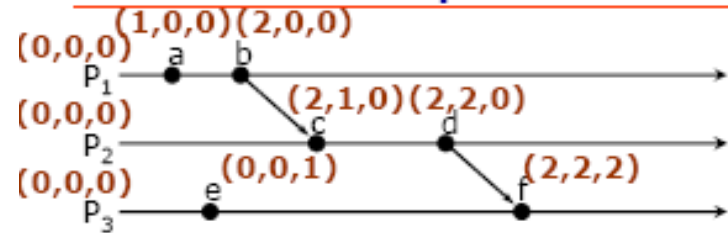
## Vector timestamps



| Event | timestamp |
|-------|-----------|
| a     | (1,0,0)   |
| b     | (2,0,0)   |
| c     | (2,1,0)   |
| d     | (2,2,0)   |
| e     | (0,0,1)   |
| f     | (2,2,2)   |

concurrent events

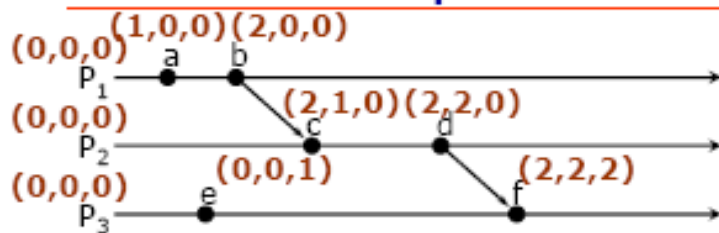
## Vector timestamps



| Event | timestamp |
|-------|-----------|
| a     | (1,0,0)   |
| b     | (2,0,0)   |
| c     | (2,1,0)   |
| d     | (2,2,0)   |
| e     | (0,0,1)   |
| f     | (2,2,2)   |

concurrent events

## Vector timestamps

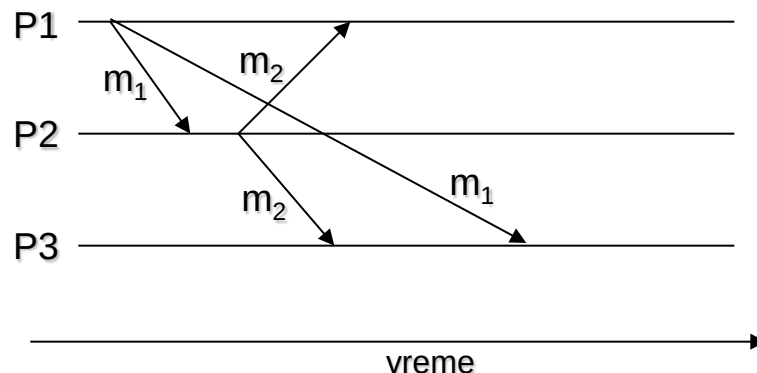


| Event | timestamp |
|-------|-----------|
| a     | (1,0,0)   |
| b     | (2,0,0)   |
| c     | (2,1,0)   |
| d     | (2,2,0)   |
| e     | (0,0,1)   |
| f     | (2,2,2)   |

concurrent events

# Primer primene: Uređenje međusobno zavisnih događaja

Vektorski časovnici se mogu iskoristiti za korektno uređenje međusobno zavisnih događaja



## \* Poruka $m_1$ se šalje svim procesima u grupi (multicast)

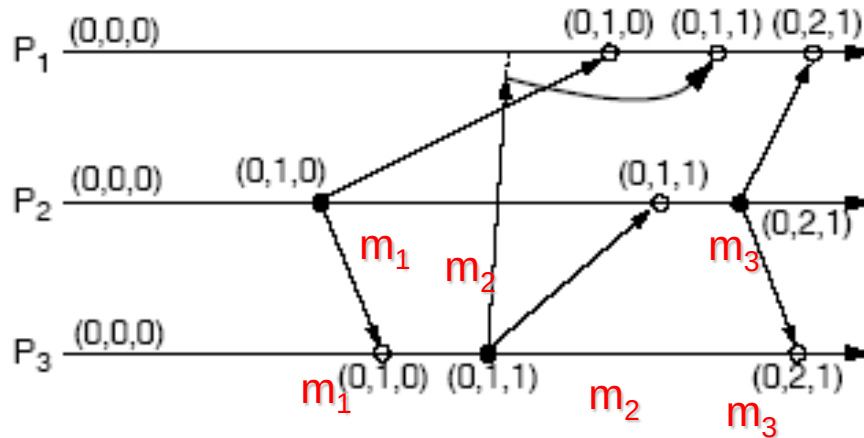
- Poruka  $m_2$  je odgovor na poruku  $m_1$  i takođe se šalje svim procesima.
- Potrebno je osigurati da svi procesi prvo procesiraju poruku  $m_1$  a zatim  $m_2$ .
  - Ako  $m_2$  stigne pre mora se baferovati dok ne stigne poruka  $m_1$
  - Uređenje zavisnih događaja
  - Vektorski časovnici se mogu iskoristiti za postizanje željenog uređenja
    - Časovnik se inkrementira samo kod slanja poruke

# Pravila

1. Pre emisije poruke  $m$ , proces  $P_i$  inkrementira svoj vektorski časovnik
  - $V_i[i] = V_i[i] + 1$
  - Markica  $tm = V_i$  se šalje sa porukom  $m$
2. Na prijemnoj strani poruka  $m$  se ne prosleđuje procesu  $P_j$  dok se ne zadovolje sledeća dva uslova
  - A)  $tm[i] = V_j[i] + 1$ 
    - Ovaj uslov obezbeđuje da proces  $P_j$  primi sve poruke koje je poslao proces  $P_i$  pre slanja poruke  $m$
    - Neka je  $V_j[i] = 5$ . Ovo znači da  $P_j$  zna da je  $P_i$  poslao 5 poruka i da od njega očekuje 6. poruku.
    - Ako je  $tm[i] = 8$  tada poruke 6,7 još nisu stigle
  - B)  $tm[k] \leq V_j[k]$ , za svako  $k \neq i$ 
    - Obezbeđuje da proces  $P_j$  primi sve poruke koje je primio  $P_i$  pre slanja poruke  $m$
3. Kada se poruka prosledi procesu, njegov lokalni časovnik se ažurira

$$V_j[i] = \max\{tm[i], V_j[i]\}, \text{ za } i, j = 1, \dots, N$$

# Primer (nast)



Poruka  $m_2$  će biti primljena u  $P_1$  sa  $tm_2=(0,1,1)$ ,

- Lokalni vektor u  $P_1$  je  $V_1=(0,0,0)$ ,
- Proveravaju se uslovi A) i B)
  - A)  $tm_2[3] = 1 = V_1[3] + 1 = 0 + 1 = 1$ ,
  - B) ali je  $tm_2[2] = 1 > V_1[2] = 0$  ( a mora biti  $<$  ili  $=$ ); ➡ poruka mora biti baferovana (proces  $P_1$  nije video poruku iz  $P_2$  koju je video  $P_3$  pre slanja poruke  $m_2$ , pa mora sačekati da primi poruku iz proces  $P_1$ )

Kada stigne  $m_1$  sa  $tm_1=(0,1,0)$ , proveravaju se uslovi A) i B)

A)  $tm_1[2] = 1 = V_1[2] + 1 = 0 + 1 = 1$

B)  $tm_1[1] = 0 = V_1[1] = 0$ ,  $tm_1[3] = 0 = V_1[3] = 0$

- Uslovi A) i B) su zadovoljeni; poruka se prosleđuje procesu, a lokalni časovnik se ažurira shodno pravilu 3, i postaje  $V_1=[0,1,0]$
- Poruka  $m_2$  sada može biti isporučena!

# Sinhronizacija procesa: Uzajamno isključivanje

✱ Pristup zajedničkim resursima (uređaji, mem. lokacije) zahteva međusobnu koordinaciju procesa da bi se ostvarilo uzajamno isključivo pravo pristupa zajedničkom resursu.

- U sistemima sa zajedničkom memorijom uzajamno isključivanje se može postići hardverskim mehanizmima (test\_and\_set) ili softverski pomoću semafora ili monitora.
- U DS uzajamno isključivanje se ne može ostvariti semaforima ili monitorima.
  - Jedini način je razmenom poruka.

✱ Algoritmi uzajamnog isključivanja u DS mogu biti

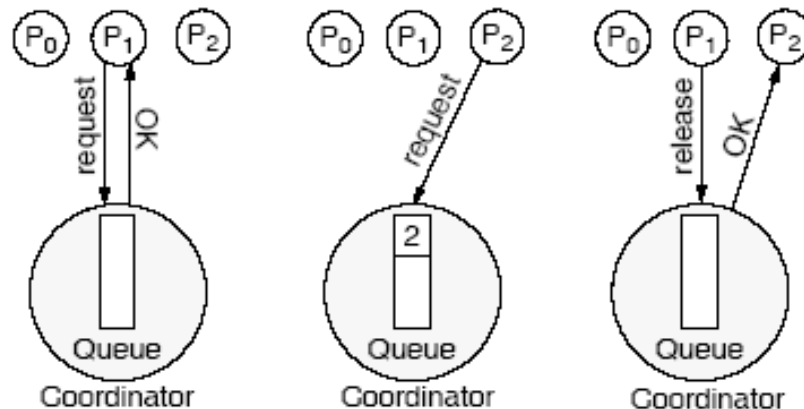
- Centralizovani
- Distribuirani
- Bazirani na žetonima (token)

# Centralizovani algoritam

\* Oponaša jednoprocesorski sistem

\* Jedan proces u DS je odabran za koordinatora

- Kada proces želi da pristupi deljivom resursu (kritičnoj sekciji) šalje zahtev koordinatoru identifikujući kritičnu sekciju kojoj želi da pristupi (po imenu ili broju).
- Ako ni jedan drugi proces nije u datoj kritičnoj sekciji, koordinator šalje dozvolu i označava da proces koristi datu KS.
- Ako je neki drugi proces u KS, koordinator ne šalje odgovor, a proces koji je uputio zahtev se blokira.
- Kada proces okonča pristup KS šalje poruku oslobađanja koordinatoru
  - Koordinator zatim može poslati dozvolu pristupa KS procesu koji je bio blokiran čekajući na ulazak u KS.



# Centralizovani algoritam (nast.)

## \* Dobre strane

- Jednostavan i lak za implementaciju
- Zahteva razmenu samo tri poruke (request, OK, release)
- Fer: svi zahtevi se opslužuju po redosledu prijema

## \* Loše strane

- Koordinator može postati usko grlo sa stanovišta performansi
- Otkaz koordinatora
  - U slučaju otkaza potrebno je izabrati novog koordinatora
- Proces ne može razlikovati otkaz koordinatora od čekanja na oslobađanje KS
  - da bi se ovaj problem prevazišlo, koordinator može poslati poruku odbijanja "permission denied"

# Distribuirani algoritam: Ricart & Agrawala

✱ Koristi multicast (grupnu komunikaciju) i logičke časovnike.

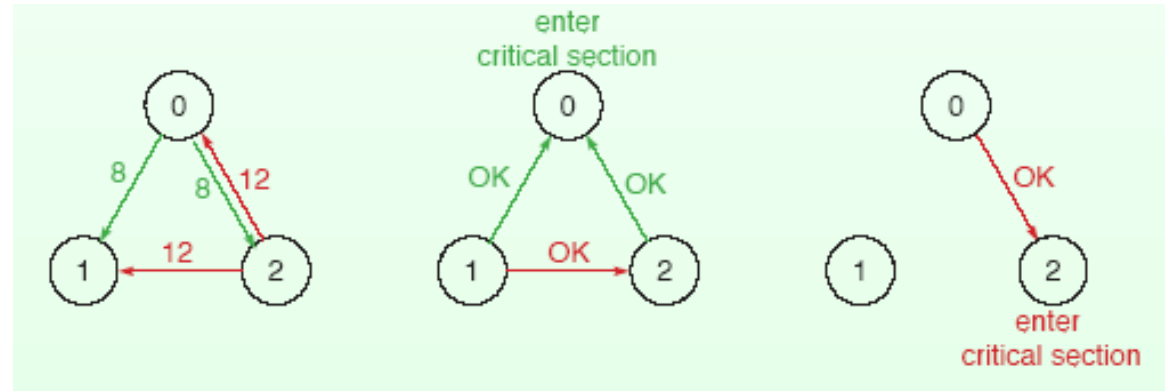
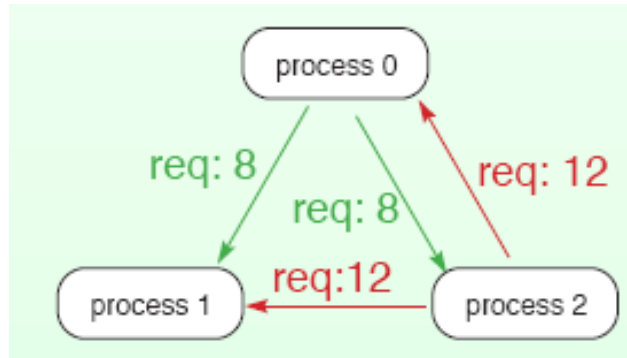
- Proces koji želi da uđe u KS

- Kreira poruku koja sadrži:
  - Identifikator (host\_ID, proces\_ID)
  - Ime resursa
  - Logički časovnik
- Šalje zahtev svim procesima u grupi
- Čeka na dozvolu od svih procesa
- Ulazi u KS (koristi resurs)

- Kada proces primi zahtev

- Ako nije zainteresovan za korišćenje resursa šalje potvrdu (OK) pošiljaocu
- Ako je prijemnik u KS, ne odgovara i smešta zahtev u svoj red čekanja
- Ako je prijemnik upravo poslao zahtev za pristup KS
  - Poredi vremenske markice poslatog i primljenog zahteva
    - » Proces sa manjom markicom pobeđuje
    - » Prijemnik šalje potvrdu (OK)
    - » Ako je prijemnik pobednik, ne šalje potvrdu već smešta zahtev u svoj red čekanja
- Kada okonča pristup KS, šalje potvrdu (OK) svim zahtevima koji su bili u redu čekanja.

# Distribuirani algoritam: Ricart & Agrawala (nast.)

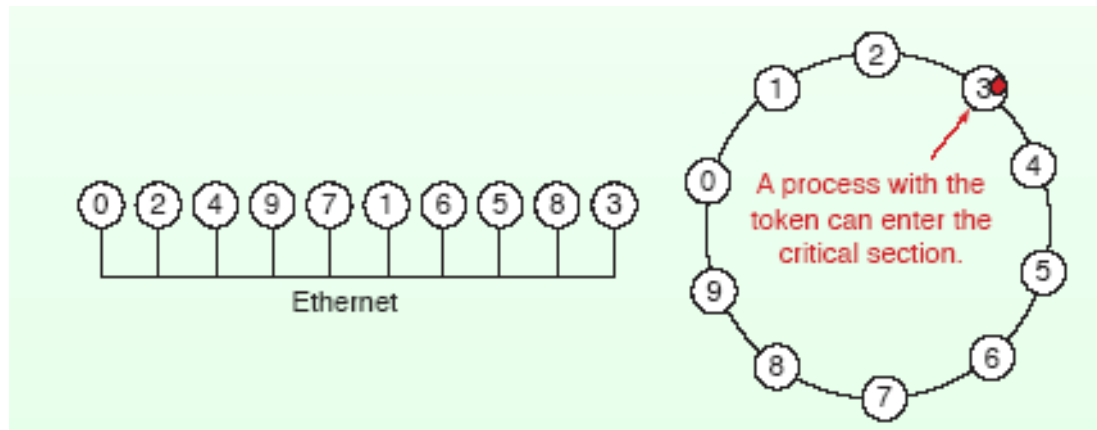


- \* Algoritam garantuje uzajamno isključivo pravo pristupa bez smrtnog zagrljaja (deadlock) i "izgladnjivanja"
- \* Algoritam zahteva razmenu  $2(n-1)$  poruka za pristup KS ( $n-1$  zahtev i  $n-1$  potvrda)
- \* Da bi se dobilo pravo pristupa resursu, potrebno je da se primi potvrda od svih procesa ( $n-1$ )
- \* Greška u bilo kom procesu blokira ceo sistem
- \* Pokazuje da je potpuno distribuirani algoritam uzajamnog isključivanja moguć.
- \* Algoritam se može popraviti tako što će proces uvek slati odgovor na zahtev (dozvolu ili odbijanje)
  - Proces koji je izdao zahtev može da koristi timeout mehanizam i da ponovi zahtev dok ne dobije odgovor ili zaključi da je proces mrtav
- \* Može se ublažiti zahtev i da se dobije potvrda od  $m > n/2$  procesa
  - U ovom slučaju proces koji je dao dozvolu nekom drugom procesu da uđe u KS ne sme dati dozvolu sledećem dok se resurs ne oslobodi.

# Algoritam sa prstenom i žetonima (token ring algoritam)

## \* Konstruiše se logički prsten dodelom rednih brojeva procesima

- Logički prsten je softverski generisan i nema nikakve veze sa fizičkim vezama između računara.
- Proces komunicira sa svojim logičkim susedom
  - Svaki proces ima adresu (identifikator) svog desnog suseda u pravcu okretanja kazaljke na satu.



# Token ring algoritam (nast.)

- \* Inicijalno proces 0 dobija žeton za resurs R.
- \* Žeton se prenosi u pravcu kazaljke na satu od procesa do procesa dok se ne dođe do procesa koji zahteva resurs R.
  - Proces koji zahteva resurs R čeka dok ne dobije žeton od svog levog suseda.
    - Kada proces dobije žeton, zadržava ga i pristupa resursu R (ulazi u KS).
    - Kada izađe iz KS prosleđuje žeton sledećem procesu u prstenu.

# Token ring algoritam (nast.)

## \* Osobine

- Samo jedan proces može posedovati žeton u jednom trenutku
  - Uzajamno isključivanje je zagarantovano
- Redosled pristupa resursu je dobro definisan
  - Svakom procesu se garantuje pristup resursu R
- Ne garantuje FIFO redosled pristupa resursu.



- Ako se žeton izgubi (npr. greška u procesu) neophodno je izvršiti rekonfiguraciju prstena da bi se isključio proces koji je otkazao.
  - Potrebno je regenerisati žeton i ponovo ga pustiti u ring (neophodno je startovati algoritam izbora – election algorithm)

# Algoritmi izbora (election algorithms)

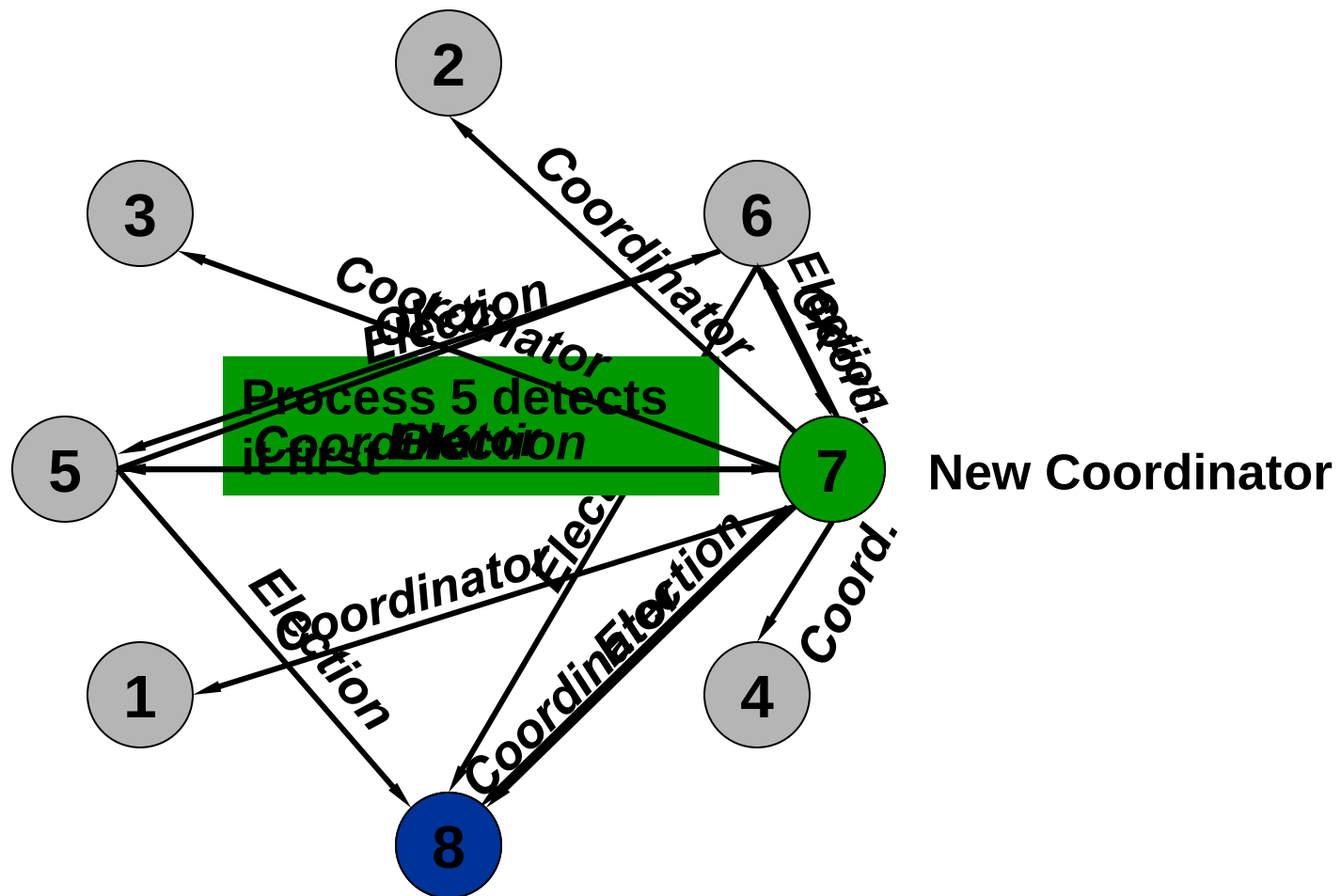
✱ Distribuirani algoritmi često zahtevaju da jedan proces bude odabran kao koordinator ili server.

- Algoritmi za selekciju koordinatora ili servera se zovu *algoritmi izbora*.
- Polazne pretpostavke
  - svaki proces ima jedinstveni identifikator (može biti kombinacija adrese računara i proces ID).
  - Svaki proces zna jedinstvene identifikatore ostalih procesa, ali ne zna da li je odgovarajući proces živ.
- Cilj algoritama izbora je da pronađu aktivni proces sa najvećim identifikatorom i proglase ga za koordinatora.
  - Svi procesi se moraju složiti oko izbora koordinatora.
- Proces izbora se obično odvija u dve faze
  - Selekcija lidera sa najvećim brojem
  - Obaveštavanje svih procesa o pobedniku

# Bully algoritam

- \* Proces,  $P_i$ , koji detektuje da koordinator ne odgovara startuje izbor.
  - Izborna poruka se šalje svim procesima sa većim identifikatorom i čeka se odgovor.
    - Ako u okviru određenog vremenskog intervala ne stigne odgovor ni od jednog procesa, to znači da ni jedan proces sa većim identifikatorom nije aktivan i pobednik je proces koji je inicirao ozbore (tj.  $P_i$ ).
      - $P_i$  obaveštava sve procese da je on koordinator
    - Ako pristigne odgovor na poruku izbora, proces  $P_i$  se povlači i čeka da dobije poruku od novog koordinatora.

# Bully Algoritam - primer



# Ring algoritam izbora

## \* Procesi formiraju logički prsten.

- Ako neki proces detektuje da koordinator ne funkcioniše, startuje algoritam izbora novog koordinatora.
  - Šalje poruku izbora (election message) sa svojim ID svom susedu u prstenu.
  - Ako sused nije aktivan poruka se prosleđuje sledećem procesu u prstenu.
  - Po prijemu poruke izbora, svaki proces joj pridodaje svoj ID i prosleđuje sledećem procesu.
  - Kada poruka stigne do procesa koji je inicirao izbore, on detektuje u poruci svoj ID.
    - Na osnovu primljene poruke proces zaključuje ko su članovi prstena
    - Šalje novu poruku "KOORDINATOR" sa brojem procesa sa najvećim identifikatorom da obavesti ostalne procese o novom koordinatoru.

